

TAREA 5.3 - Desarrollo de una función factorial usando TDD

Manuel Parrado Torres

12/02/2026



Índice

1	Fase RED – Test que falla	2
2	Fase GREEN – Mínimo código para que pase el test	3
3	Fase REFACTOR – Mejora del código	5

1 Fase RED – Test que falla

Escribe primero un test con pytest (por ejemplo, para factorial(0)).

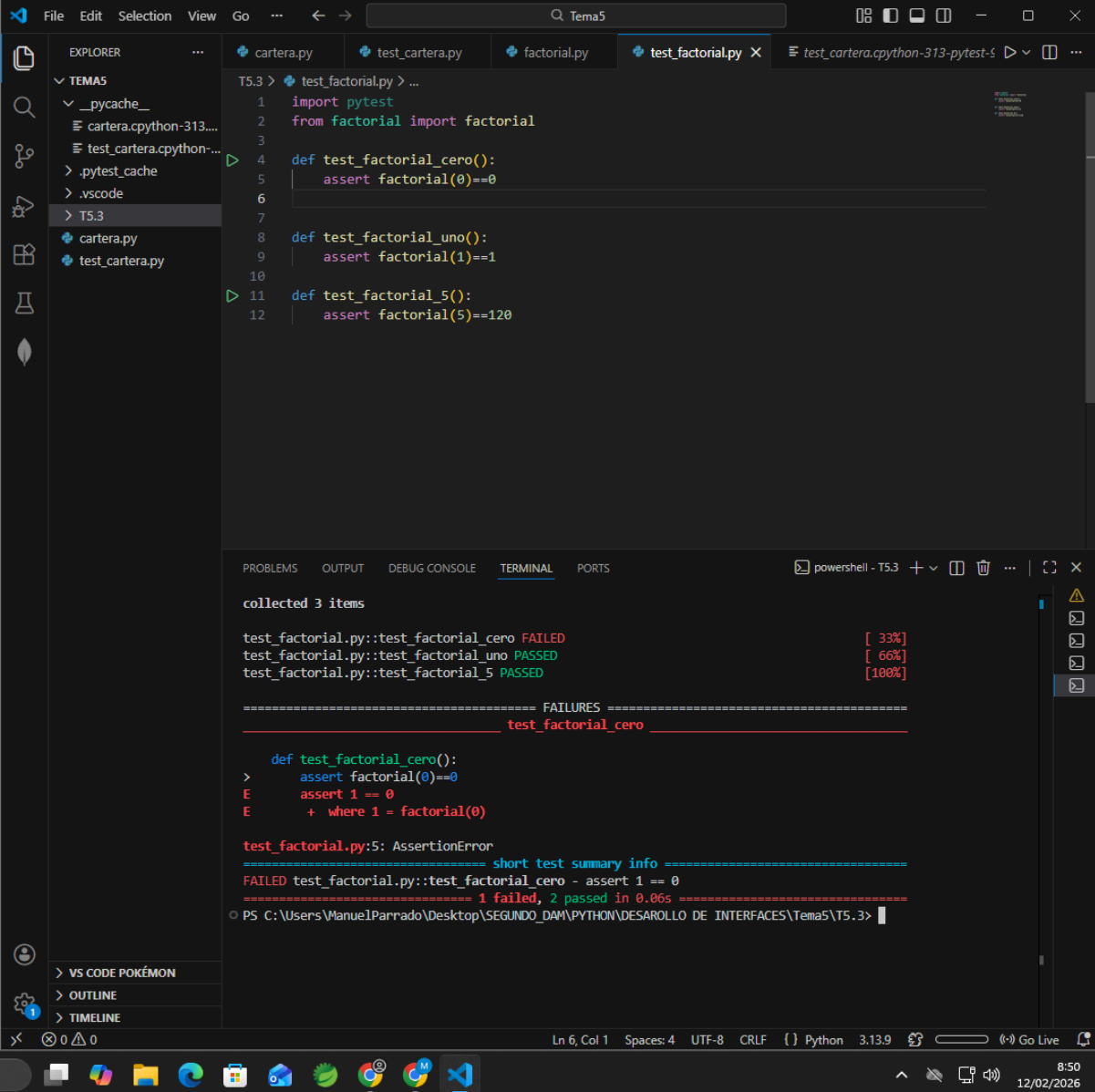
Ejecuta pytest.

El test debe fallar.

📸 Captura obligatoria en el PDF:

Código del test

Resultado del test fallando en consola



The screenshot shows the Visual Studio Code interface with a Python project. The Explorer sidebar on the left shows a file structure with 'Tema5' containing 'cartera.py', 'test_cartera.py', 'factorial.py', and 'test_factorial.py'. The main editor displays the code for 'test_factorial.py', which includes three test functions: 'test_factorial_cero()', 'test_factorial_uno()', and 'test_factorial_5()'. The 'test_factorial_cero()' function contains an assertion that fails because 'factorial(0)' is 1, not 0. The bottom panel shows the 'TERMINAL' output, which displays the pytest results: 'collected 3 items', 'test_factorial.py::test_factorial_cero FAILED [33%]', 'test_factorial.py::test_factorial_uno PASSED [66%]', and 'test_factorial.py::test_factorial_5 PASSED [100%]'. Below this, a detailed failure report for 'test_factorial_cero' is shown, indicating an 'AssertionError' at line 5. The status bar at the bottom indicates the current file is 'Ln 6, Col 1' and the Python version is '3.13.9'.

```
1 import pytest
2 from factorial import factorial
3
4 def test_factorial_cero():
5     assert factorial(0)==0
6
7
8 def test_factorial_uno():
9     assert factorial(1)==1
10
11 def test_factorial_5():
12     assert factorial(5)==120
```

```
collected 3 items

test_factorial.py::test_factorial_cero FAILED [ 33%]
test_factorial.py::test_factorial_uno PASSED [ 66%]
test_factorial.py::test_factorial_5 PASSED [100%]

===== FAILURES =====
_____ test_factorial_cero _____

    def test_factorial_cero():
>     assert factorial(0)==0
E       assert 1 == 0
E       + where 1 = factorial(0)

test_factorial.py:5: AssertionError

===== short test summary info =====
FAILED test_factorial.py::test_factorial_cero - assert 1 == 0
===== 1 failed, 2 passed in 0.06s =====
PS C:\Users\ManuelParrado\Desktop\SEGUNDO_DAM\PYTHON\DESAROLLO DE INTERFACES\Tema5\T5.3>
```

2 Fase GREEN – Mínimo código para que pase el test

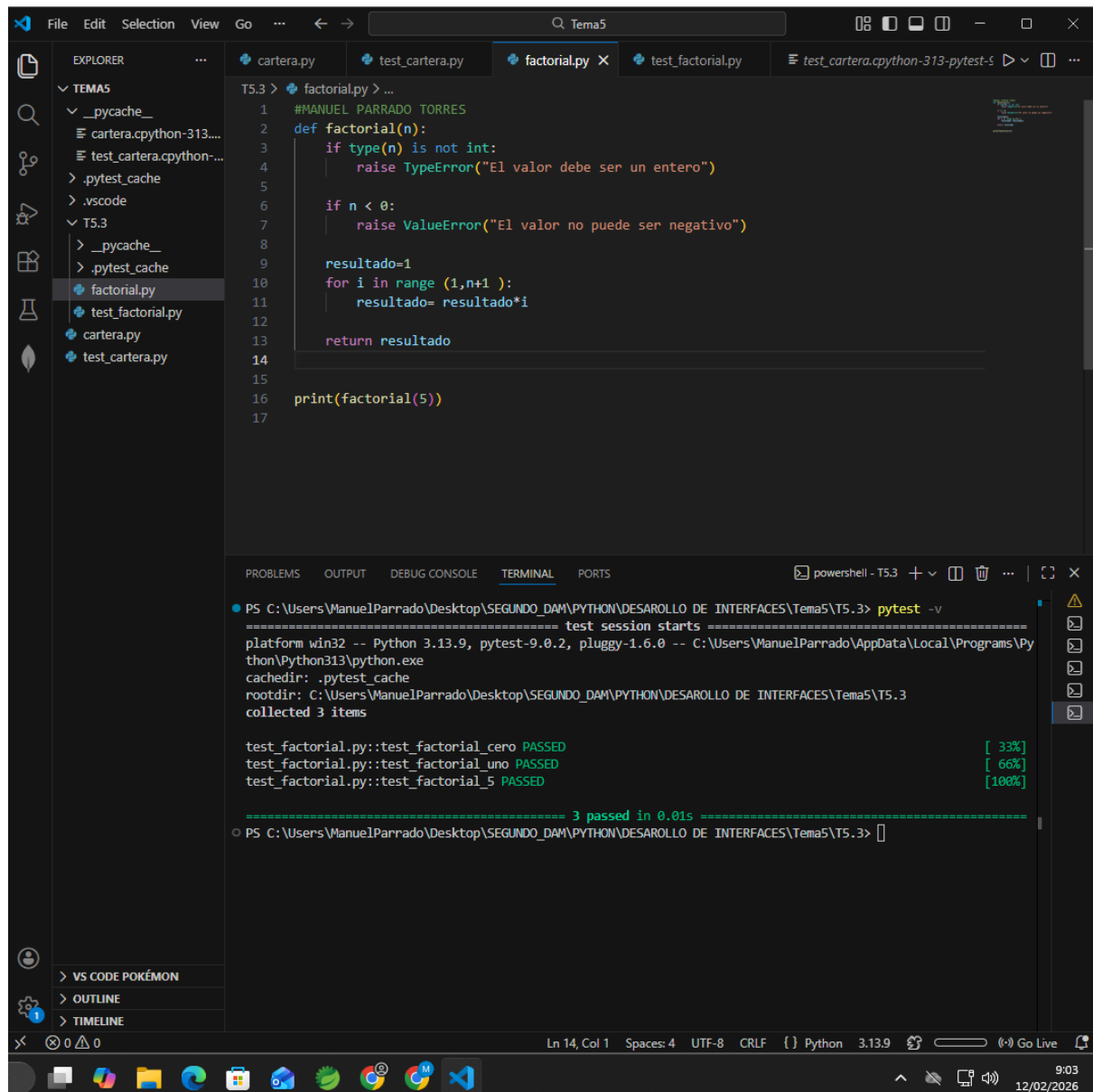
Implementa el mínimo código posible para que el test pase.

Ejecuta pytest de nuevo.

📸 Captura obligatoria en el PDF:

Código mínimo de la función

Resultado del test pasando



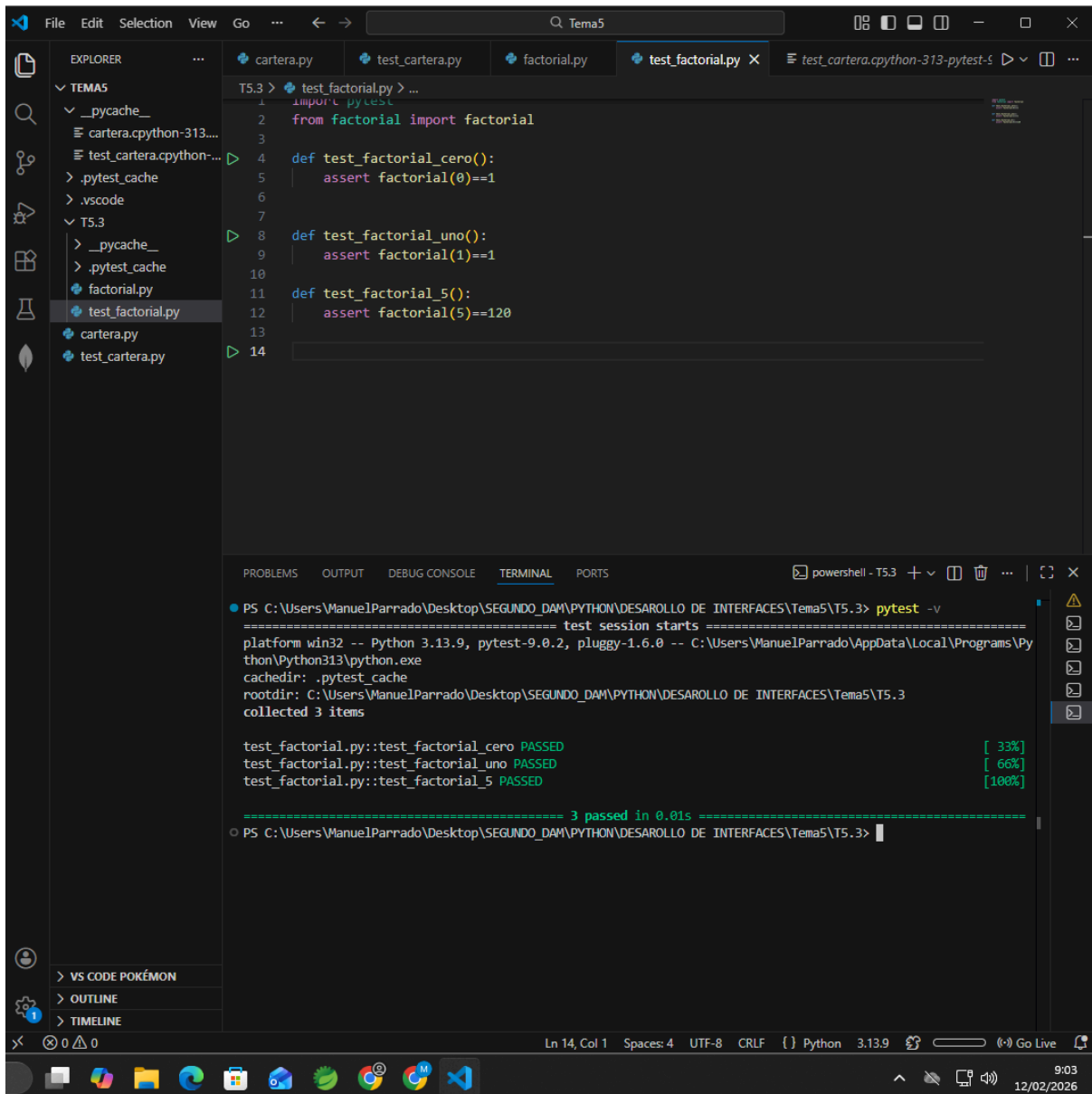
The screenshot shows the Visual Studio Code editor with a file explorer on the left and a terminal at the bottom. The file explorer shows a project structure with folders like 'TEMAS', '._pycache_', and 'T5.3'. The 'T5.3' folder contains files like 'factorial.py', 'test_factorial.py', 'cartera.py', and 'test_cartera.py'. The 'factorial.py' file is open in the editor, showing a Python function 'factorial(n)' that checks for integer input and non-negative values, then calculates the factorial using a loop. The terminal shows the output of running 'pytest -v' on the 'test_factorial.py' file, indicating that all three tests passed successfully.

```
1 #MANUEL PARRADO TORRES
2 def factorial(n):
3     if type(n) is not int:
4         raise TypeError("El valor debe ser un entero")
5
6     if n < 0:
7         raise ValueError("El valor no puede ser negativo")
8
9     resultado=1
10    for i in range (1,n+1 ):
11        resultado= resultado*i
12
13    return resultado
14
15
16 print(factorial(5))
17
```

```
PS C:\Users\ManuelParrado\Desktop\SEGUNDO_DAM\PYTHON\DESAROLLO DE INTERFACES\Tema5\T5.3> pytest -v
===== test session starts =====
platform win32 -- Python 3.13.9, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\ManuelParrado\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\ManuelParrado\Desktop\SEGUNDO_DAM\PYTHON\DESAROLLO DE INTERFACES\Tema5\T5.3
collected 3 items

test_factorial.py::test_factorial_cero PASSED [ 33%]
test_factorial.py::test_factorial_uno PASSED [ 66%]
test_factorial.py::test_factorial_5 PASSED [100%]

===== 3 passed in 0.01s =====
PS C:\Users\ManuelParrado\Desktop\SEGUNDO_DAM\PYTHON\DESAROLLO DE INTERFACES\Tema5\T5.3>
```



3 Fase REFACTOR – Mejora del código

Añade nuevos tests de forma incremental:

factorial(1)

factorial(5)

Casos de error (negativo y no entero)

Refactoriza la función para que quede clara y general.

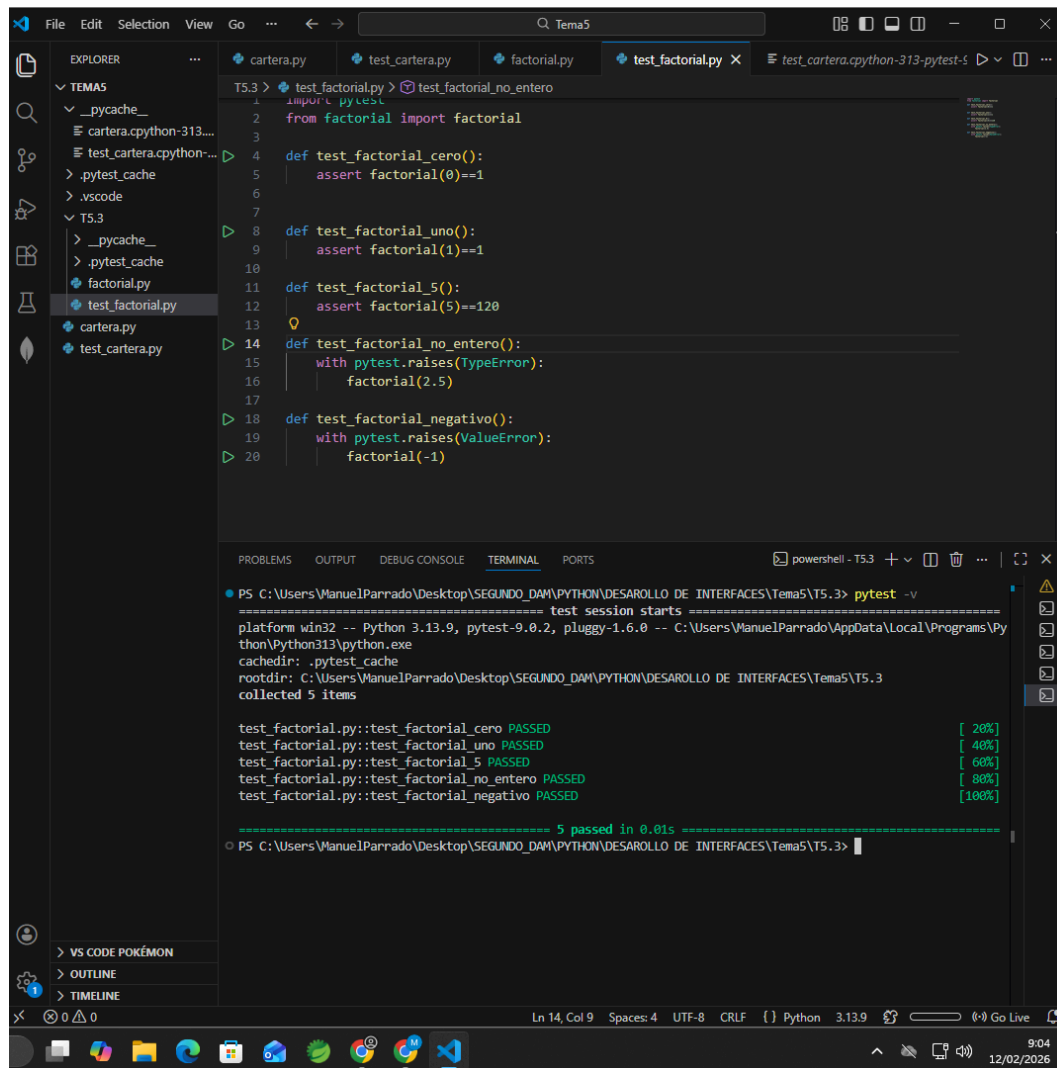
Los tests deben seguir pasando tras cada cambio.

📸 Capturas obligatorias en el PDF:

Nuevos tests añadidos

Código refactorizado

Ejecución final de pytest con todos los tests en verde



The screenshot shows the Visual Studio Code interface. The Explorer sidebar on the left shows a project structure with folders like 'TEMAS', '._pycache_', and files like 'cartera.py', 'test_cartera.py', 'factorial.py', and 'test_factorial.py'. The main editor window displays the content of 'test_factorial.py'. The code includes imports for 'pytest' and 'factorial', and five test functions: 'test_factorial_cero()', 'test_factorial_uno()', 'test_factorial_5()', 'test_factorial_no_entero()', and 'test_factorial_negativo()'. The 'test_factorial_no_entero()' test uses 'pytest.raises(TypeError)' to expect an exception from 'factorial(2.5)', and 'test_factorial_negativo()' uses 'pytest.raises(ValueError)' for 'factorial(-1)'. Below the editor, the 'TERMINAL' panel shows the output of running 'pytest -v'. The output indicates that all five tests passed successfully, with progress markers [20%], [40%], [60%], [80%], and [100%]. The final summary line states '5 passed in 0.01s'.

```
File Edit Selection View Go ... Tema5
EXPLORER
TEMAS
  ._pycache_
    cartera.cpython-313...
    test_cartera.cpython-...
  .pytest_cache
  .vscode
  T5.3
    ._pycache_
    .pytest_cache
    factorial.py
    test_factorial.py
    cartera.py
    test_cartera.py

T5.3 > test_factorial.py > test_factorial_no_entero
1 import pytest
2 from factorial import factorial
3
4 def test_factorial_cero():
5     assert factorial(0)==1
6
7
8 def test_factorial_uno():
9     assert factorial(1)==1
10
11
12 def test_factorial_5():
13     assert factorial(5)==120
14
15 def test_factorial_no_entero():
16     with pytest.raises(TypeError):
17         factorial(2.5)
18
19 def test_factorial_negativo():
20     with pytest.raises(ValueError):
21         factorial(-1)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\ManuelParrado\Desktop\SEGUNDO_DAM\PYTHON\DESAROLLO DE INTERFACES\Tema5\T5.3> pytest -v
===== test session starts =====
platform win32 -- Python 3.13.9, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\ManuelParrado\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\ManuelParrado\Desktop\SEGUNDO_DAM\PYTHON\DESAROLLO DE INTERFACES\Tema5\T5.3
collected 5 items

test_factorial.py::test_factorial_cero PASSED [ 20%]
test_factorial.py::test_factorial_uno PASSED [ 40%]
test_factorial.py::test_factorial_5 PASSED [ 60%]
test_factorial.py::test_factorial_no_entero PASSED [ 80%]
test_factorial.py::test_factorial_negativo PASSED [100%]

===== 5 passed in 0.01s =====
PS C:\Users\ManuelParrado\Desktop\SEGUNDO_DAM\PYTHON\DESAROLLO DE INTERFACES\Tema5\T5.3>
```

